Institución Universitaria Digital de Antioquia — IU Digital

Programación para Análisis de Datos

Grupo PREICA2602B020048

Evidencia de aprendizaje 1

Análisis y herramientas de extracción de datos

Web scraping de un artículo de Amazon con BeautifulSoup, Selenium y Scrapy

Integrantes:

Simón Castaño Ríos

Juan Pablo Gonzalez Gil

Luisa Fernanda Gómez Quiroz

Docente: Andrés Felipe Palacio

Octubre de 2026

1. Introducción

Las tiendas en línea publican a diario información valiosa sobre sus productos: precios, calificaciones, número de opiniones y comentarios de los compradores. Esa información está pensada para leerse en el navegador, no para descargarse en bloque; para convertirla en datos analizables se usa el web scraping, que consiste en programas que solicitan una página, interpretan su estructura HTML y guardan los datos de interés en un formato tabular.

En esta evidencia se aplica la metodología de scraping de la Unidad 1 a un artículo de Amazon usando las tres herramientas más conocidas de Python: BeautifulSoup, Selenium y Scrapy. Las tres se aplican sobre la misma ficha de producto para comparar qué datos obtiene cada una, cuánto tardan y cómo reaccionan a las verificaciones anti-bot del sitio. Los datos obtenidos (CSV y Excel) son la base de las evidencias de las unidades 2 y 3 y de la actividad de cierre.

Inicialmente se evaluaron SHEIN y Mercado Libre. SHEIN redirige cualquier petición automatizada a una verificación de seguridad (/risk/challenge) y Mercado Libre exige iniciar sesión para ver productos, por lo que se eligió Amazon, cuya ficha de producto sí es accesible.

2. Descripción de la página y del artículo a analizar

2.1 La página: Amazon

Amazon es la empresa de comercio electrónico más grande del mundo. Fue fundada por Jeff Bezos en 1994 en Seattle (Estados Unidos) como librería en línea y hoy vende millones de productos de terceros y propios. Desde Colombia se puede comprar en amazon.com con envío internacional: el sitio muestra los precios convertidos a pesos colombianos (COP) e indica si el artículo se envía al país.

Cada producto tiene una ficha identificada por un código ASIN (Amazon Standard Identification Number) con título, precio, calificación promedio, número de calificaciones, características, distribución de estrellas y reseñas destacadas. La ficha se entrega en el HTML inicial, lo que permite extraerla con una petición HTTP, aunque Amazon aplica verificaciones (captcha) cuando detecta tráfico automatizado.

2.2 El artículo seleccionado

Atributo	Valor obtenido
Título	HSHFKLH 3 piezas de arte de pared de lienzo de mujeres afroamericanas de 12
ASIN	B0D21SJ6BR
URL	https://www.amazon.com/dp/B0D21SJ6BR
Tienda	HSHFKLH
Categoría	Hogar y Cocina › Obras de Arte y Material Decorativo › Pósteres
Precio	$94.633 COP (precio convertido por Amazon el 24/10/2026)
Calificación	4.8 de 5 (104 calificaciones)
Disponibilidad	Disponible
Descripción	Set de 3 lienzos con retratos coloridos de mujeres afroamericanas en estilo graffiti/abstracto; tamaños de 12×16 y 16×24 pulgadas, con marco de madera, listo para colgar.

Se eligió un cuadro decorativo porque no depende de tallas ni colores: tiene un precio y un conjunto de opiniones únicos, lo que simplifica la comparación entre métodos y el seguimiento del precio en el tiempo.

3. Descripción del tema de interés

El tema de interés es la **relación entre precio, calificación y opinión de los compradores en artículos de decoración de bajo costo vendidos en Amazon.**

En esta primera práctica el foco es metodológico: construir y validar la extracción de los datos de un artículo (precio, calificación, número de calificaciones, distribución de estrellas, características y reseñas) con tres herramientas distintas. En las siguientes unidades se ampliará a varios cuadros similares para comparar precios, analizar qué valoran los compradores (colores, tamaño, calidad, embalaje) y observar la evolución del precio.

4. Objetivos

¿Por qué analizar este artículo y esta empresa? Amazon es el mayor referente del comercio electrónico y una opción de compra real para los consumidores colombianos; sus fichas concentran precio y opinión de cientos de compradores. Analizar un artículo de decoración permite entender cómo se percibe la calidad de un producto económico y es útil para un consumidor o un emprendedor local que compite con esos precios. Además, es un buen caso para comparar herramientas de scraping frente a un sitio grande con defensas anti-bot.

4.1 Objetivo general

Aplicar la metodología de web scraping con BeautifulSoup, Selenium y Scrapy para extraer y estructurar la información pública de un artículo de Amazon, comparando la eficacia de cada herramienta.

4.2 Objetivos específicos

1. Identificar en el HTML de la ficha los selectores CSS de título, precio, calificación, número de calificaciones, tienda, disponibilidad, categoría, características, distribución de estrellas y reseñas.
2. Implementar la extracción con requests + BeautifulSoup.
3. Implementar la extracción con Selenium, controlando un navegador Chrome real.
4. Implementar un spider con Scrapy que respete robots.txt y exporte los datos en JSON.
5. Comparar los tres métodos (campos obtenidos, tiempo, consistencia, bloqueos) y guardar los datos en CSV y Excel.

5. Metodología empleada de scraping

5.1 Especificación de la solución

Se siguió el ciclo de scraping: (1) inspección de la página con las herramientas de desarrollador del navegador para ubicar los selectores; (2) solicitud de la página; (3) extracción con selectores CSS/XPath; (4) limpieza (texto a número, eliminación de textos de accesibilidad); (5) almacenamiento en JSON, CSV y Excel; y (6) validación comparando los resultados de los tres métodos. Los selectores y las funciones de limpieza están en un módulo común para que la comparación sea justa: solo cambia la herramienta.

Archivo	Función
comun.py	URL, encabezados, selectores CSS y funciones de limpieza.
scraper_beautifulsoup.py	Método 1: requests + BeautifulSoup.
scraper_selenium.py	Método 2: Selenium + Chrome.
scraper_scrapy.py	Método 3: Spider de Scrapy.
comparar_resultados.py	Tabla comparativa y tabla de reseñas (CSV y Excel).
Castano_Simon_Gonzalez_JuanPablo_Gomez_LuisaFernanda_Analizando_EA1.ipynb	Cuaderno Jupyter que ejecuta todo y genera los gráficos.

Dato	Selector CSS en la ficha de Amazon
Título	#productTitle
Precio	#corePrice_feature_div .a-price .a-offscreen
Calificación	#acrPopover (atributo title)
N.º de calificaciones	#acrCustomerReviewText
Tienda	#bylineInfo
Disponibilidad	#availability
Categoría	#wayfinding-breadcrumbs_feature_div
Características	#feature-bullets li
Distribución de estrellas	#cm_cr_dp_d_rating_histogram a[aria-label]
Reseñas	[data-hook="review"] → reviewTitle, review-star-rating, review-date, reviewText

5.2 Método 1: BeautifulSoup

Se hace una petición GET con la librería requests, enviando un User-Agent de navegador y el idioma es-CO. BeautifulSoup (con el parser lxml) recorre el HTML con select() y select_one().

El HTML descargado se guarda en data/pagina_amazon.html. Es el método más simple y rápido, pero no ejecuta JavaScript.

5.3 Método 2: Selenium

Selenium WebDriver abre Google Chrome (el driver lo instala Selenium Manager), carga la ficha y espera con WebDriverWait a que aparezca #productTitle. Luego baja hasta la sección de opiniones para que carguen, lee los elementos con find_elements(By.CSS_SELECTOR, …) y toma una captura de pantalla. Si Amazon pide una verificación, el script no la evade: espera a que la persona la complete en la ventana de Chrome.

5.4 Método 3: Scrapy

Se define el spider ProductoAmazonSpider con ROBOTSTXT_OBEY = True (la ruta /dp/ está permitida en el robots.txt de Amazon), DOWNLOAD_DELAY = 2 s y una petición a la vez. Usa los selectores propios de Scrapy: XPath para el título y la calificación, y CSS (::text, ::attr) para el resto. Los resultados se exportan con FEEDS a data/producto_scrapy.json. Si recibe un captcha, lo registra en el log y procesa la copia del HTML guardada por el método 1; el campo fuente del resultado lo indica.

5.5 Métricas de éxito

Métrica	Definición	Meta
Completitud	Campos principales obtenidos (de 7)	7/7 en cada método
Consistencia	Los tres métodos devuelven los mismos valores	100 %
Reseñas	Reseñas extraídas con título, estrellas, fecha y texto	≥ 5
Tiempo	Segundos desde la petición hasta el dato limpio	< 30 s por método
Bloqueos	Captchas detectados y documentados, sin evadirlos	100 % detectados

5.6 Consideraciones éticas

- Se respeta robots.txt y se hace una sola petición por método, con retardo.
- Solo se recogen datos públicos del producto; de las reseñas no se guardan nombres de usuarios.
- No se evaden captchas ni controles de acceso; el uso es exclusivamente académico.

6. Resultados

6.1 Comparación de los tres métodos

Método	Fuente de la página	Tiempo (s)	Campos	Caract.	Reseñas
BeautifulSoup	en vivo	2.7	7/7	5	8
Selenium	en vivo	8.8	7/7	5	8
Scrapy	HTML guardado (captcha en vivo)	4.6	7/7	5	8

Los tres métodos obtuvieron los 7 campos principales con valores idénticos (precio $94.633 COP, calificación 4.8, 104 calificaciones), las 5 características y las 8 reseñas destacadas. BeautifulSoup fue el más rápido (2.7 s) y Selenium el más lento (8.8 s), porque debe abrir un navegador completo y esperar a que cargue la página.

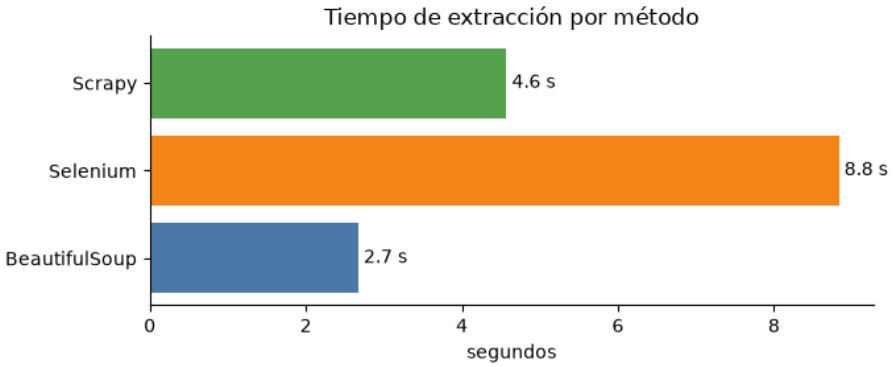


Figura 1. Tiempo de extracción por método.

6.2 Hallazgos sobre las verificaciones anti-bot

- **Scrapy:** en todas las ejecuciones Amazon le respondió con un captcha (HTTP 200 con una página de verificación de unos 3.700 caracteres). El spider lo detectó y procesó el HTML guardado.
- **Selenium:** en algunas ejecuciones Amazon mostró una pantalla intermedia («Continue shopping») en lugar del producto. El script la detecta y, con la ventana visible, espera a que la persona haga clic.
- **Reseñas:** en algunas visitas Amazon muestra «Identifícate para ver las opiniones de los clientes» y no incluye el texto de las reseñas; la calificación y la distribución de estrellas sí aparecen siempre.
- **SHEIN y Mercado Libre:** SHEIN redirigió las peticiones a /risk/challenge (captcha) y Mercado Libre a una página de inicio de sesión, por lo que no fueron viables sin cuenta.

6.3 Datos del artículo

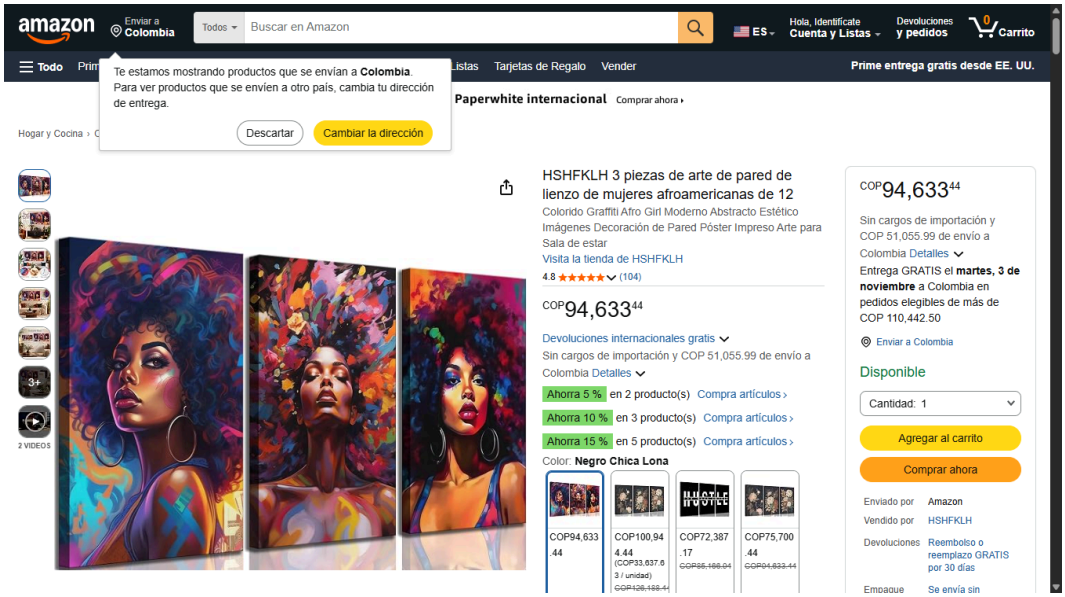


Figura 2. Ficha del producto capturada por Selenium.

El artículo tiene una percepción muy positiva: 91 % de las calificaciones son de 5 estrellas y 96 % son de 4 o 5; solo 3 % son de 1 o 2 estrellas.

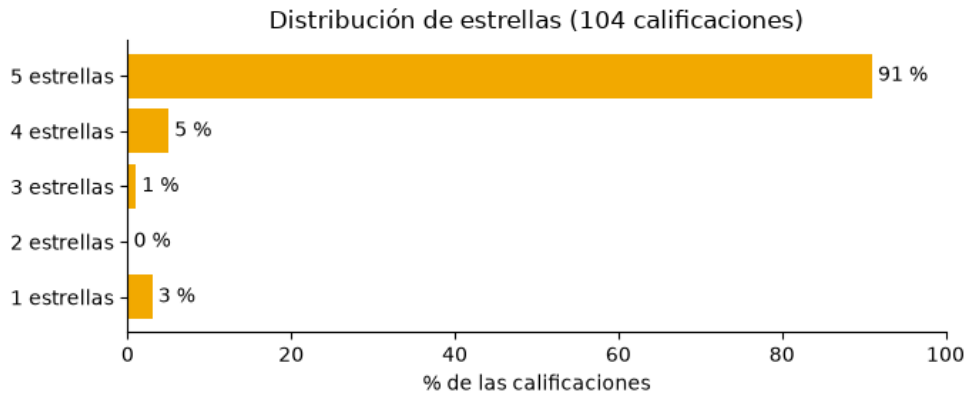


Figura 3. Distribución de estrellas.

Título de la reseña	Estrellas	Fecha
Love it!	5	2 de agosto de 2026
Black Art	5	24 de julio de 2025
Gorgeous 🥰	5	5 de mayo de 2026
I am in Love with these paintings!	5	25 de abril de 2025
I love it!	5	31 de agosto de 2025
Beautiful art work	5	19 de mayo de 2025
Beautiful	5	5 de enero de 2025
Beautiful Artwork	5	1 de septiembre de 2025

Las palabras más frecuentes en las reseñas (colors, love, beautiful, vibrant, look) muestran que los compradores valoran sobre todo los colores vivos y la apariencia; también mencionan el tamaño y la calidad.



Figura 4. Palabras más frecuentes en las reseñas destacadas.

7. Conclusiones

- **BeautifulSoup** fue la opción más sencilla y rápida: como Amazon entrega la ficha en el HTML inicial, una petición y unos selectores CSS bastaron para obtener todos los datos.
- **Selenium** obtuvo los mismos datos pero tardó más del triple; su ventaja es que ve la página como un usuario (JavaScript, contenido diferido) y permite completar a mano una verificación.
- **Scrapy** ofrece la mejor estructura para escalar (robots.txt, retardos, exportación automática), pero Amazon bloqueó su petición en vivo; separar la descarga del análisis permitió usarlo igualmente.
- **Validar es tan importante como extraer:** detectar captchas y páginas de inicio de sesión evitó guardar datos vacíos o incorrectos.
- **Sobre el artículo:** cuesta $94.633 COP, tiene 4.8/5 con 104 calificaciones y opiniones centradas en colores y apariencia, lo que indica alta satisfacción para un producto económico.
- **Siguiente paso:** ampliar el scraping a varios cuadros similares para comparar precio y calificación, y repetirlo en el tiempo para seguir la evolución del precio.

8. Bibliografía

Amazon. (2026). HSHFKLH 3 piezas de arte de pared de lienzo de mujeres afroamericanas [Página de producto]. Recuperado el 24 de octubre de 2026, de https://www.amazon.com/dp/B0D21SJ6BR

Mitchell, R. (2018). Web scraping with Python: Collecting more data from the modern web (2.ª ed.). O'Reilly Media.

Richardson, L. (2024). Beautiful Soup documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Scrapy developers. (2025). Scrapy documentation. https://docs.scrapy.org/

Selenium Project. (2025). The Selenium browser automation project: Documentation. https://www.selenium.dev/documentation/

Stack Overflow contributors. (s. f.). Aprendizaje BeautifulSoup [Libro electrónico]. Material de apoyo del curso.

Stack Overflow contributors. (s. f.). Aprendizaje Scrapy [Libro electrónico]. Material de apoyo del curso.

Stack Overflow contributors. (s. f.). Aprendizaje Selenium [Libro electrónico]. Material de apoyo del curso.

Anexo A. Ejecución y fragmentos del código

El código completo está en la carpeta scraping/ y en el cuaderno Castano_Simon_Gonzalez_JuanPablo_Gomez_LuisaFernanda_Analizando_EA1.ipynb. Ejecución desde la carpeta scraping/:

```
python -m pip install -r requirements.txt
python scraper_beautifulsoup.py
python scraper_selenium.py
python scraper_scrapy.py
python comparar_resultados.py
```

BeautifulSoup (scraper_beautifulsoup.py):

```
respuesta = requests.get(URL, headers=HEADERS, timeout=30)
soup = BeautifulSoup(respuesta.text, "lxml")
titulo = soup.select_one("#productTitle").get_text(strip=True)
precio = soup.select_one("#corePrice_feature_div .a-price .a-offscreen").get_text()
resenas = soup.select('[data-hook="review"]')
```

Selenium (scraper_selenium.py):

```
driver = webdriver.Chrome(options=opciones)
driver.get(URL)
WebDriverWait(driver, 20).until(
    EC.presence_of_element_located((By.CSS_SELECTOR, "#productTitle")))
precio = driver.find_element(By.CSS_SELECTOR, "#corePrice_feature_div .a-offscreen")
```

Scrapy (scraper_scrapy.py):

```
class ProductoAmazonSpider(scrapy.Spider):
    name = "producto_amazon"
    start_urls = [URL]
    custom_settings = {"ROBOTSTXT_OBEY": True, "DOWNLOAD_DELAY": 2}

    def extraer(self, response, fuente):
        titulo = response.xpath("normalize-space(//span[@id='productTitle'])").get()
        calificacion = response.xpath("//*[@id='acrPopover']/@title").get()
        precio = response.css("#corePrice_feature_div .a-price .a-offscreen ::text").get()
```